

LAB #5 Directory Access, Hard Links, and Symbolic Links

1. *Apply directory access system calls to display and interpret directory contents.*
2. *Analyze file attributes using stat() and lstat() system calls.*
3. *Differentiate between hard links and symbolic links using inode analysis.*
4. *Demonstrate the ability to create, verify, and manage file links in a Linux environment.*

Pre-Lab Theory:

SYNOPSIS

```
#include <dirent.h>
#include <dirent.h>
#include <sys/stat.h>

char *getcwd(char *buf, size_t size);
DIR *opendir(const char *dirname);
struct dirent *readdir(DIR *dirp);
int closedir(DIR *dirp);
void rewinddir(DIR *dirp);
int lstat(const char *restrict path, struct stat *restrict buf);
int stat(const char *restrict path, struct stat *restrict buf);
```

The struct stat structure, which is defined in sys/stat.h, contains at least the following members.

```
dev_t st_dev; /* device ID of device containing file */
ino_t st_ino; /* file serial number */
mode_t st_mode; /* file mode */
nlink_t st_nlink; /* number of hard links */
uid_t st_uid; /* user ID of file */
gid_t st_gid; /* group ID of file */
off_t st_size; /* file size in bytes (regular files) */
/* path size (symbolic links) */
time_t st_atime; /* time of last access */
time_t st_mtime; /* time of last data modification */
time_t st_ctime; /* time of last file status change */
```

LAB TASKS

Task 1: Apply the `getcwd()` system call to obtain and display the current working directory path.

```
#include <stdio.h>
#include <unistd.h>

int main() {
    char cwd[1024];
    // char *getcwd(char *buf, size_t size);
    // On success → returns the same pointer (buf)
    // On failure → returns NULL and sets an error code
/* char *result = getcwd(cwd, sizeof(cwd));
   if (result != NULL)
       printf("Current Working Directory: %s\n", cwd);
   else
       perror("getcwd() error");
*/
    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("Current Working Directory: %s\n", cwd);
    else
        perror("getcwd() error");
    return 0;
}
```

Write the full directory path displayed by your program.

```
muhammad-ahmad@latitude-7490:~/os/lab5$ ./task1
Current Working Directory: /home/muhammad-ahmad/os/lab5
```

Task 2: Implement and demonstrate directory traversal using `opendir()`, `readdir()`, and `closedir()`.

```
#include <stdio.h>
#include <dirent.h> // for DIR, struct dirent, opendir, readdir, closedir

int main() {
    DIR *dir; // pointer to a directory stream , tell pwd
    struct dirent *entry; // structure pointer , tell directory insides

    dir = opendir("."); // from current directory
    if (dir == NULL) { // check for valid dir
        perror("opendir() error");
        return 1;
    }

    printf("Directory contents:\n");
    // struct dirent *readdir(DIR *dirp);
```

```

        while ((entry = readdir(dir)) != NULL) {
            printf("%s\n", entry->d_name); // printing only name from
            struct , the struct have other info also
        }

        closedir(dir);
        return 0;
    }

```

Output:

muhammad-ahmad@latitude-7490:~/os/lab5\$./task2

Directory contents:

```

task6
task5.c
task1
task1.c
task4.c
test
test.txt
task3
task2
hardlink_test.txt
.
task5
task6.c
softlink_test.txt
task3.c
task2.c
..

```

Task 3: Differentiate between files and directories using file type macros with stat().

```
#include <stdio.h>
```

```
#include <dirent.h>
```

```
#include <sys/stat.h> // S_ISDIR()
```

```

int main() {
    DIR *dir;
    struct dirent *entry;
    struct stat fileStat;
    dir = opendir("."); // path to open
    while ((entry = readdir(dir)) != NULL)
    {
        stat(entry->d_name, &fileStat);
        if (S_ISDIR(fileStat.st_mode)) // printing directories
            printf("%s --> Directory\n", entry->d_name);
        else // printing files
            printf("%s --> File\n", entry->d_name);
    }
}

```

```

        closedir(dir); return 0;
    }

```

- Paste the program output showing which items are identified as files or directories.

```

muhammad-ahmad@latitude-7490:~/os/lab5$ ./task3
task6 --> File
task1 --> File
test --> Directory
test.txt --> File
task3 --> File
task2 --> File
hardlink_test.txt --> File
. --> Directory
task5 --> File
task6.c --> File
softlink_test.txt --> File
task3.c --> File
task2.c --> File
.. --> Directory

```

- Write the names of at least two directories detected by your code.

```

.
..
test

```

Task 4: Apply the `stat()` system call to extract and analyze metadata such as permissions, owner, and link count.

```

#include <stdio.h>
#include <sys/stat.h>

```

```

int main(int argc, char *argv[]) {
    struct stat fileStat;
    // Check if filename is provided
    if (argc < 2) {
        printf("Usage: %s <filename>\n", argv[0]);
        return 1;
    }
    // Apply stat() to the file given in argv[1]
    if (stat(argv[1], &fileStat) < 0) {
        perror("stat() error");
        return 1;
    }
    // Display file metadata
    printf("File: %s\n", argv[1]);
    printf("Inode: %ld\n", fileStat.st_ino);
    printf("Permissions: %o\n", fileStat.st_mode & 0777);
    printf("Owner UID: %d\n", fileStat.st_uid);
    printf("Group GID: %d\n", fileStat.st_gid);
    printf("Size: %ld bytes\n", fileStat.st_size);
    printf("Hard Links: %ld\n", fileStat.st_nlink);
}

```

```
    return 0;
}
```

Paste the output showing metadata for test.txt.

```
muhammad-ahmad@latitude-7490:~$ ./task4 Downloads
File: Downloads
Inode: 11272269
Permissions: 755
Owner UID: 1000
Group GID: 1000
Size: 4096 bytes
Hard Links: 6
```

Task 5: *Demonstrate the creation of hard and symbolic links and evaluate their structural differences using inode numbers.*

```
#include <stdio.h>
#include <unistd.h> #include
<sys/stat.h>

int main() {
    link("test.txt", "hardlink_test.txt");
    symlink("test.txt", "softlink_test.txt");
    printf("Hard and soft links created successfully.\n");
    struct stat hardStat, softStat;
    stat("hardlink_test.txt", &hardStat);
    lstat("softlink_test.txt", &softStat);
    printf("Hard Link inode: %ld\n", hardStat.st_ino);
    printf("Soft Link inode: %ld\n", softStat.st_ino);
    return 0;
}
```

- **Paste the output showing successful creation of both links.**

```
muhammad-ahmad@latitude-7490:~/os/lab5$ ./task5
Hard and soft links created successfully.
```

- **Record the inode numbers displayed for original file, hard link, and soft link.**

```
Original inode: 11551967
Hard Link inode: 11551967
Soft Link inode: 11552908
```

- **Write which link remains accessible after deleting the original file.**

After deleting the original file, only the **hard link** remains accessible, while the **soft link** becomes a broken pointer.

Task 6: Extend the Task 2 to implement and demonstrate recursive directory traversal using `opendir()`, `readdir()`, and `closedir()` to display the contents of the current directory and all its subdirectories up to the last level in the hierarchy.

```
#include <stdio.h>
#include <dirent.h>
#include <sys/stat.h>
#include <string.h>

// Recursive function to list all files and directories
void listDirectory(const char *path, int depth) {
    DIR *dir;
    struct dirent *entry;
    struct stat fileStat;
    char fullPath[1024];

    dir = opendir(path);
    if (dir == NULL) {
        perror("opendir() error");
        return;
    }

    while ((entry = readdir(dir)) != NULL) {
        // Skip current and parent directory entries
        if (strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name, "..") == 0)
            continue;

        // Build full path (parent path + "/" + entry name)
        snprintf(fullPath, sizeof(fullPath), "%s/%s", path, entry->d_name);

        // Get file information
        if (stat(fullPath, &fileStat) == -1) {
            perror("stat() error");
            continue;
        }

        // Print indentation (tabs) according to depth level
        for (int i = 0; i < depth; i++)
            printf("\t");

        // Check file type and print
        if (S_ISDIR(fileStat.st_mode)) {
            printf(" %s (Directory)\n", entry->d_name);
            // Recursive call for the subdirectory
            listDirectory(fullPath, depth + 1);
        } else {
            printf(" %s (File)\n", entry->d_name);
        }
    }

    closedir(dir);
}

int main() {
    printf("Directory Tree Structure:\n");
    listDirectory(".", 0);
    return 0;
}
```

Paste the output showing full directory structure printed by your program (including subdirectories).

```
muhammad-ahmad@latitude-7490:~/os/lab5$ ./task6
```

```
Directory Tree Structure:
```

```
task6 (File)
task5.c (File)
task1 (File)
task1.c (File)
task4.c (File)
test (Directory)
  1.dat (File)
  test1 (Directory)
    1.txt (File)
    2.txt (File)
  2.dat (File)
test.txt (File)
task3 (File)
task2 (File)
hardlink_test.txt (File)
task5 (File)
task6.c (File)
softlink_test.txt (File)
task3.c (File)
task2.c (File)
```

Write the name of the deepest subdirectory found during execution.

```
test1
```